



tailwindcss

`text-8xl text-gray-950 tracking-tighter text-balance`

Rapidly build modern websites without ever leaving your HTML.

`text-lg text-gray-950 font-medium`

A utility-first CSS framework packed with classes like `flex`, `pt-4`, `text-center` and `rotate-90` that can be composed to build any design, directly in your markup.

<https://tailwindcss.com/>

Agenda



1. Inleiding: CSS paradigma's
2. Architectuur van Tailwind
3. Installatie: Vite + tailwind 4
4. More Utility-first
5. Mobile first Responsive Design
6. Theming & dark mode
7. @layer & @apply
8. Verder...
9. daisyUI

Ondersteunend materiaal



Video

-  [Tailwind CSS v4 Full Course 2025](https://www.youtube.com/watch?v=...)
 - Download code en ontwikkel mee:
<https://gist.github.com/adrianhajdin/b974da379cb372e46ab29deec366c5c3>
 - Wij gebruiken pnpm/vite

Wat is een tailwind?

  <https://tailwindcss.com>

- CSS framework
- Style HTML met een bibliotheek van utility CSS classes
 - vele kleine CSS klassen met elk één duidelijke taak
 - combineer flexibel voor het gewenste resultaat
 - CSS klassen werken op een hoger niveau dan inline styles
 - configureerbaar
- CSS uitbreidingen: varianten, Extra CSS functies, directives...
 - een browser begrijpt geen tailwind
 - moet eerst omgezet worden naar CSS

Inleiding: CSS paradigma's



Wat is er mis met gewone CSS ?

→ De syntax van traditionele css is declaratief

*d.w.z. **handgeschreven** stylesheets met classnamen en selectors gekoppeld aan visuele eigenschappen*



veel controle



grote, moeilijk onderhoudbare bestanden.

Waarom Tailwind?

Vanilla CSS → flexibel, maar complex op schaal. Moeilijk onderhoud

Component libraries: snel, maar weinig uniek.

- Bootstrap, Material UI of Ant Design voorgedefinieerde UI-componenten.
- versnellen de ontwikkeling
- generieke user interfaces (Cookie cutter design)

 **Tailwind:** flexibel, performant en schaalbaar

Waarom Tailwind?

- **Sneller bouwen**

Geen class-namen bedenken, geen selectors, geen CSS-bestanden
→ direct stylen in HTML = sneller resultaat

- **Veilig wijzigingen maken**

Utility aanpassen = impact op 1 element
→ geen onverwachte breuken elders

- **Makkelijk onderhoud**

Oud project aanpassen = element zoeken + classes wijzigen
→ geen vergeten, complexe CSS-logica

Waarom Tailwind?

- **Hergebruik & portabiliteit**

Structuur + styling samen zijn universeel

→ UI-blokken eenvoudig kopiëren (zelfs tussen projecten)



Let op

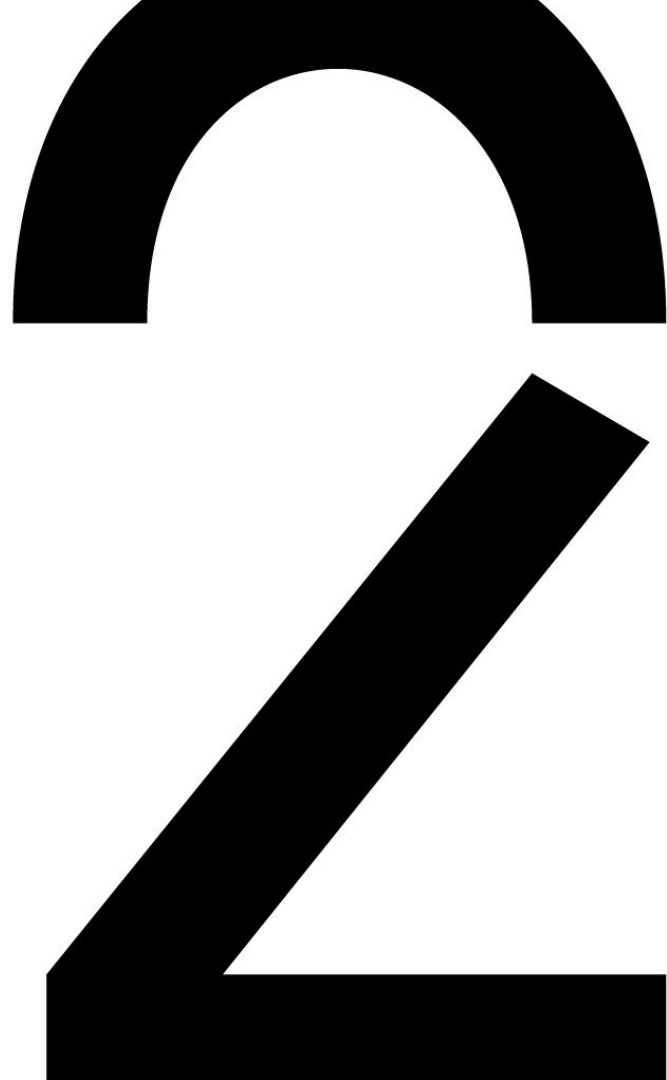
vind een balans: als je steeds dezelfde combinatie van klassen kopieert, gebruik je beter CSS regels en klassen! (DRY principe)

- **CSS groeit niet eindeloos**

Utilities zijn herbruikbaar

→ geen lineaire CSS-explosie per feature

Architectuur van Tailwind

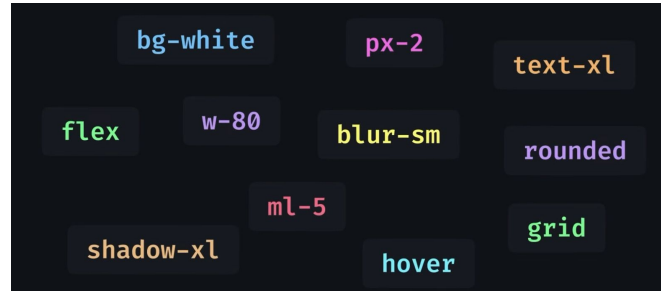


Tailwind CSS introduceert een utility-first paradigma :



- styling opgebouwd uit kleine, herbruikbare klassen.
- combineren classes rechtstreeks in HTML.

Composition over inheritance.



👉 bouw gedrag door dingen te combineren i.p.v. ze van elkaar te laten over-erven.

CSS uitbreidingen en extra class names

- extra varianten. Voorbeeld `lg`: wordt alleen toegepast op large displays
- extra functies. Voorbeeld: `--spacing`
- extra directieven
 - `@theme`: definieer een thema voor je site
 - `@apply`: definieer klassen op basis van bestaande klassen
 - `@layer`: bepaal de volgorde waarin CSS wordt toegepast (cascading)
 - ...
- Zet om naar CSS met vite plugin

Utility-first **!= =** Inline style

- **Ontwerpen met beperkingen**

Geen “magic numbers” → je kiest uit een vast **design system**
→ **consistente** kleuren, spacing en typografie

- **Responsive design**

Inline styles  geen media queries
Tailwind  **sm:**, **md:**, **lg:** utilities

- **States & interacties**

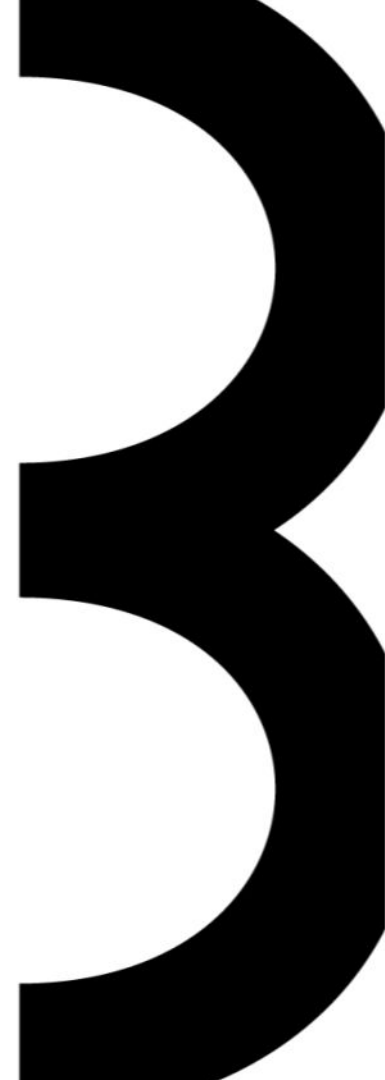
Inline styles  géén **hover** / **focus**
Tailwind  **hover:**, **focus:** varianten

- **Schaalbaar & onderhoudbaar**

Kleine utilities combineren → duidelijk, voorspelbaar, geen CSS-overerving



Installatie: Vite + tailwind 4



Installatie

[Official Vite + Tailwind installation guide](#)

```
> pnpm create vite tailwind-first --template vanilla-ts
> cd tailwind-first
> pnpm i -D tailwindcss @tailwindcss/vite
```

vite.config.ts

```
import { defineConfig } from 'vite'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    tailwindcss(),
  ],
})
```

app.css

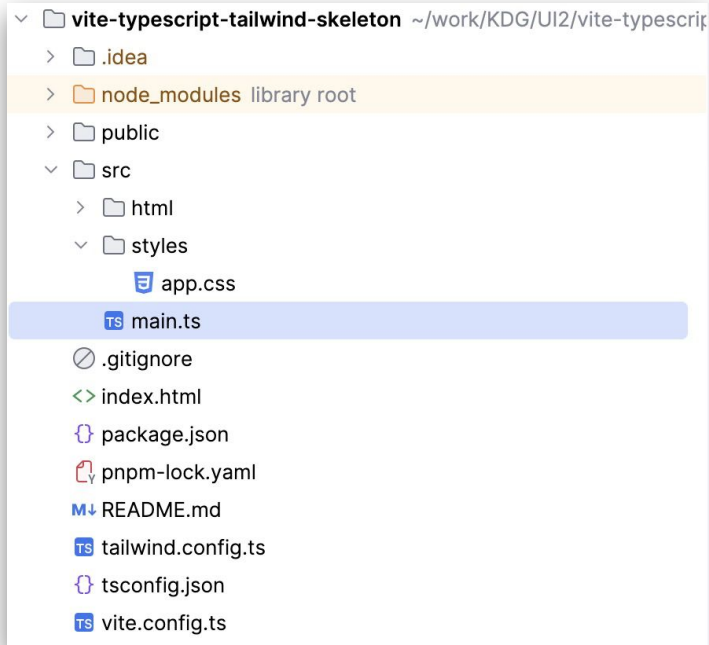
```
@import "tailwindcss";
```

```
import "tailwindcss" in app.css
+ import app.css in main.ts
+ import main.ts in index.html
```

tailwindcss() plugin om standaard CSS te genereren

Template

Vite + Typescript+ Tailwind skeleton Project Webstorm Setup



tailwind + vite hot reload
= DX magic 🪄 🎩

```
vite-typescript-tailwind-skeleton on master via v24.13.0 on ...  
> pnpm dev  
  
> vite-ts-tailwind-skeleton@0.0.1 dev /Users/seIsk/work/KDG/UI2  
> vite  
  
VITE v7.3.1 ready in 1000 ms  
  
→ Local: http://localhost:5173/ 🌐 Start Debugging in Browser  
→ Network: use --host to expose  
→ press h + enter to show help
```


Hot reloading Developer Experience:

```
1 <div class="flex flex-col items-center p-7 rounded-2xl">
2   <div>
3     
4   </div>
5   <div class="flex">
6     <span class="text-2xl font-medium">Class Warfare</span>
7     <span>The Anti-Patterns</span>
8     <span class="flex">
9       <span>No. 4</span>
10      <span>•</span>
11      <span>2025</span>
12    </span>
13  </div>
14 </div>
```

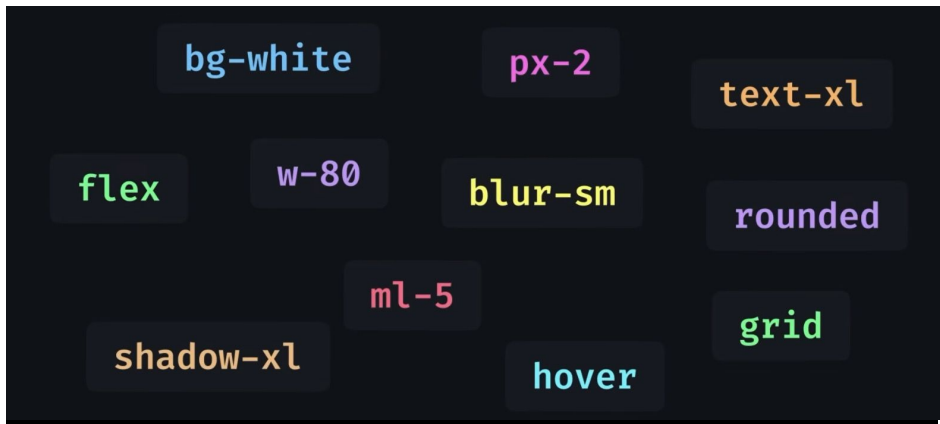


Class Warfare

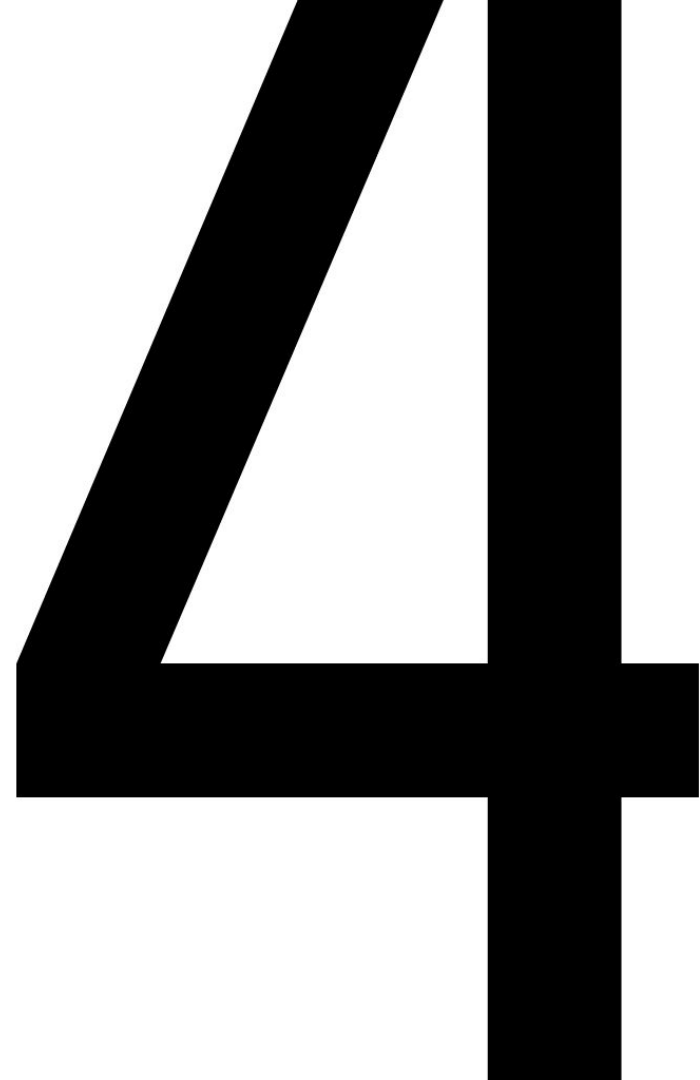
The Anti-Patterns

No. 4-2025

tailwind + vite hot reload
= DX magic 🪄🎩



More utility-first



Tailwind compiler

Just-In-Time (JIT) compiler.

- bundelt enkel de CSS-regels die daadwerkelijk in het project gebruikt worden.
- class in HTML aanwezig? → Tailwind genereert automatisch de bijbehorende CSS.

```
<div class="flex justify-center items-center">  
  <h2>Hi KdG</h2>  
</div>
```

*classes worden automatisch
gedetecteerd via de vite plugin.
Ook in via typescript
gegenereerde HTML!*

Try it out in the playground

<https://play.tailwindcss.com/2GxDWvID4m>

// generated css

```
.flex {  
  display: flex;  
}  
.items-center {  
  align-items: center;  
}  
.justify-center {  
  justify-content: center;  
}
```

Tailwind compiler

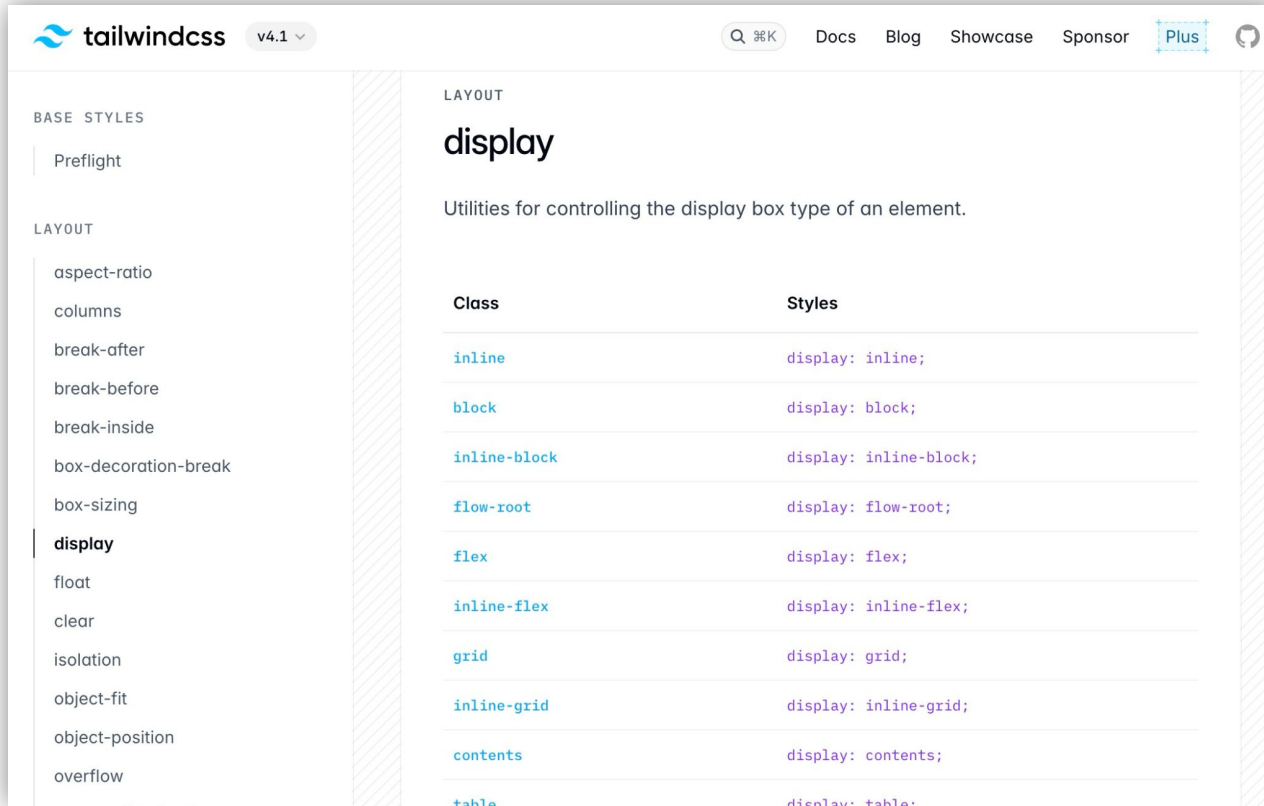
- **Atomic CSS:** elke utility staat voor een styling-eigenschap.
- **JIT:** on-demand generatie van styles.
- CSS-variabelen zorgen voor modulariteit.
- Ongebruikte utilities worden automatisch verwijderd.

bijvoorbeeld:

- ``m-*`` staat voor margin,
- ``p-*`` voor padding.
- ``flex``, ``justify-center`` en ``items-center`` regelen layout.
- ``bg-*`` en ``text-*`` bepalen kleuren.
- ``rounded-*`` zorgt voor afgeronde hoeken.

Tailwind docs

- [Tailwind v4 docs](#) utilities overzicht



The screenshot shows the Tailwind CSS v4.1 documentation page for the 'display' utility. The page has a header with the Tailwind CSS logo, version 'v4.1', and navigation links: 'Docs', 'Blog', 'Showcase', 'Sponsor', 'Plus', and a search icon. The left sidebar contains a 'BASE STYLES' section with 'Preflight' and a 'LAYOUT' section with various utilities. The 'display' utility is highlighted in the sidebar. The main content area is titled 'LAYOUT display' and includes a description: 'Utilities for controlling the display box type of an element.' Below this is a table listing the classes and their corresponding styles.

Class	Styles
<code>inline</code>	<code>display: inline;</code>
<code>block</code>	<code>display: block;</code>
<code>inline-block</code>	<code>display: inline-block;</code>
<code>flow-root</code>	<code>display: flow-root;</code>
<code>flex</code>	<code>display: flex;</code>
<code>inline-flex</code>	<code>display: inline-flex;</code>
<code>grid</code>	<code>display: grid;</code>
<code>inline-grid</code>	<code>display: inline-grid;</code>
<code>contents</code>	<code>display: contents;</code>
<code>table</code>	<code>display: table;</code>

flex-basis
flex-direction
flex-wrap
flex
flex-grow
flex-shrink
order
grid-template-columns
grid-column
grid-template-rows
grid-row
grid-auto-flow
grid-auto-columns
grid-auto-rows
gap
justify-content
justify-items
justify-self
align-content
align-items
align-self
place-content
place-items
place-self

Flexbox & Grid

Flexbox wordt gebruikt voor lineaire layouts (rijen of kolommen).

bv. navigatiebalken, formulieren en dynamische inhoud.

Grid is geschikt voor tweedimensionale layouts.
bv. dashboards of afbeelding galerijen.

- Flex: flex, flex-col, justify-*, items-*
- Grid: grid, grid-cols-3, gap-4

Puur CSS

css/puur.css

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}  
  
.container-heading {  
  font-size: 24px;  
  color: blue;  
}
```

slides/puur-css.html

```
<div class="container">  
  <h1 class="container-heading">  
    Hello World  
  </h1>  
</div>
```

tailwind

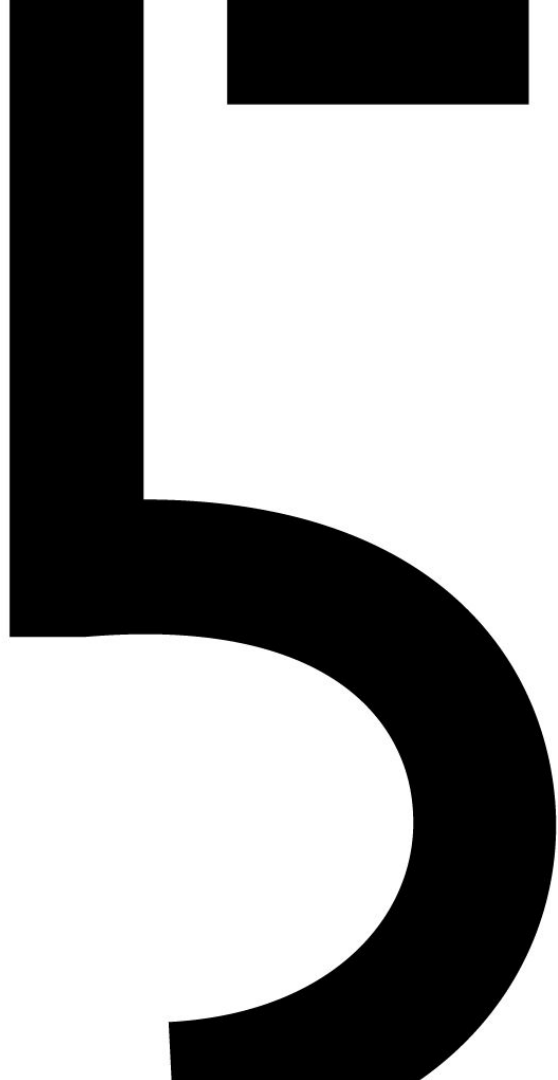
Importeer tailwind in je CSS



slides/tailwind.html

```
<div class="flex justify-center  
  items-center">  
  <h1 class="text-2xl text-blue-500">  
    Hello World  
  </h1>  
</div>
```

Varianten en Responsive Design



Varianten "Variant"

- `variant:tailwind-class`

- CSS pseudo-classes

- `status` `hover:` `focus:` `active:` ...
- `plaats` `first:` `last:` `odd:` `even:` `nth-*` `empty:` `required:` `disabled:` `has-*` ...
- combineer met not, voorbeeld: `not-focus:`

- CSS pseudo-elements `selection:` `first-line:` ...

- feature queries `dark:(in dark mode)` `print:(when printing)`...

Met de @variant directieve kan je je eigen varianten maken

slides/variants.html

```
<button class="p-2 bg-blue-500 hover:bg-blue-600  
  hover:dark:bg-blue-400">
```

Hover me

```
</button>
```

Je kan meerdere varianten combineren



Responsieve varianten

- breakpoint variant om toe te passen op viewports met grotere breedte

```
<!-- row on display >= md -->
<div class="flex flex-col md:flex-row"></div>

<div class="hidden md:block">
hide on display smaller than md
</div>

<div class="max-md:xl:hidden">
hide below md and above xl
</div>
```

- Je kan ook @sm... varianten toepassen ten opzichte van [@container HTML elementen](#) (interessant voor componenten)

Tailwind default breakpoints

Prefix	Min Width (>=)	Max Width (<)	Typical Devices
sm	40rem (640px)	<code>max-sm</code> → 40rem	Large phones
md	48rem (768px)	<code>max-md</code> → 48rem	Tablets
lg	64rem (1024px)	<code>max-lg</code> → 64rem	Small laptops
xl	80rem (1280px)	<code>max-xl</code> → 80rem	Desktops
2xl	96rem (1536px)	<code>max-2xl</code> → 96rem	Large screens

Mobile First responsive design



Don't use `sm:` to target mobile devices

HTML

```
<!-- This will only center text on screens 640px and wider, not on small screens -->  
<div class="sm:text-center"></div>
```



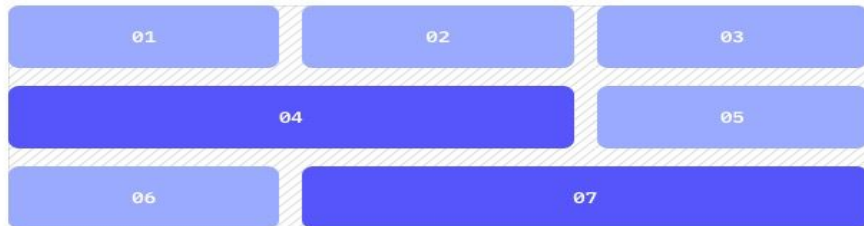
Use unprefixed utilities to target mobile, and override them at larger breakpoints

HTML

```
<!-- This will center text on mobile, and left align it on screens 640px and wider -->  
<div class="text-center sm:text-left"></div>
```

Responsieve grid met grid en met flexbox

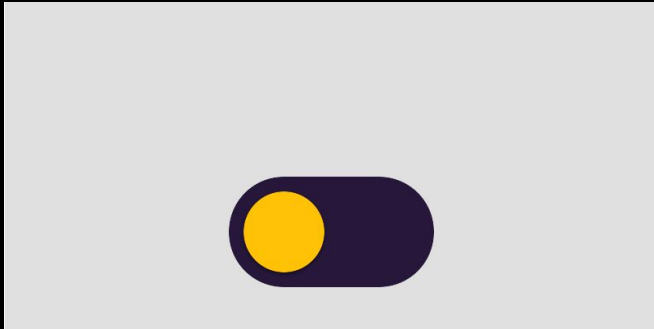
```
<div class="grid grid-cols-3 lg:grid-cols-4 gap-4">  
  <div class="...">01</div>  
  <div class="...">02</div>  
  <div class="...">03</div>  
  <div class="col-span-2 ...">04</div>  
  <div class="...">05</div>  
  <div class="...">06</div>  
  <div class="col-span-2 ...">07</div>  
</div>
```



```
<div class="flex ...">  
  <div class="w-1/2 ...">w-1/2</div>  
  <div class="w-1/2 ...">w-1/2</div>  
</div>  
<div class="flex ...">  
  <div class="w-2/5 ...">w-2/5</div>  
  <div class="w-3/5 ...">w-3/5</div>  
</div>
```



Theming





Kleuren

- Het default theme bevat een kleurenpalet
- Combineer met een utility klasse prefix (voorbeeld **text-**)

```
<h1 class="bg-gray-100 text-blue-800">Hello Tailwind</h1>
<div id="app"></div>
<script type="module">
/body>
tml>
```

text-blue-200	color: oklch(88.2% 0.059 254.128);
text-blue-300	color: oklch(80.9% 0.105 251.813);
text-blue-400	color: oklch(70.7% 0.165 254.624);
text-blue-500	color: oklch(62.3% 0.214 259.815);
text-blue-600	color: oklch(54.6% 0.245 262.881);
text-blue-700	color: oklch(48.8% 0.243 264.376);
text-blue-800	color: oklch(42.4% 0.199 265.638);
text-blue-900	color: oklch(37.9% 0.146 265.522);
text-blue-950	color: oklch(28.2% 0.091 267.935);
text-shadow-black	--tw-text-shadow-color: color-mix(in ...
text-shadow-blue-50	--tw-text-shadow-color: color-mix(i...

Webstorm popover toont CSS definitie van tailwind classes

orange-400



Utility

Description

`bg-*`

Sets the **background color** of an element

`text-*`

Sets the **text color** of an element

`decoration-*`

Sets the **text decoration color** of an element

`border-*`

Sets the **border color** of an element

Hello Tailwind



Je eigen @theme kleuren

```
@import "tailwindcss";  
  
@theme {  
  --color-lime-*: initial;  
  --color-deep-purple: #414387;  
  --color-vibrant-purple: #764abc;  
  --color-lavender: #eae3f5;  
  --color-light-purple: #baa2dc;  
  --color-gray-50: oklch(0.984 0.003 247.858);  
  --color-gray-100: oklch(0.968 0.007 247.896);  
  --color-gray-200: oklch(0.929 0.013 255.508);  
  --color-gray-300: oklch(0.869 0.022 252.894);  
  --color-gray-400: oklch(0.704 0.04 256.788);  
  --color-gray-500: oklch(0.554 0.046 257.417);  
  --color-gray-600: oklch(0.446 0.043 257.281);  
  --color-gray-700: oklch(0.372 0.044 257.287);  
  --color-gray-800: oklch(0.279 0.041 260.031);  
  --color-gray-900: oklch(0.208 0.042 265.755);  
  --color-gray-950: oklch(0.129 0.042 264.695);  
}
```

verwijderd alle lime kleuren uit default thema.

--color-: initial; verwijderd alle kleuren*

Je eigen tinten grijs

M oklch: intuïtiever alternatief voor hsv kleuren



Design tokens

- De utilities die de basis karakteristieken van het thema definiëren, zoals kleur, noemt men design tokens.
- Je kan CSS variabelen via `@theme` veranderen/toevoegen
 - `--color-* --spacing --font-* ...`

app.css

```
@import "tailwindcss";

@theme {
  --font-poppins: Poppins, sans-serif;
}
```

HTML

```
<h1 class="font-poppins">This headline will use Poppins.</h1>
```



Design tokens

- nog voorbeelden van gebruik tailwind klassen:
 - typografie: `text-{size}`, `font-{weight}`, `text-{align}`
 - spacing: `p-4` (padding 4), `mt-8` (margin top 8), `w-full` (width), `h-10` (height)
 - `--spacing` eenheid is default `0.25 rem` (4px)
 - flex en grid
 - ...

Arbitrary values & properties

[Square bracket notation]

- In de meeste gevallen kan je een UI bouwen met **design tokens** (spacing scale, colors, font sizes).
Maar soms heb je **pixel-perfect control** nodig.

```
<div class="top-[117px]"></div>
```

Dit werkt zoals inline styles, **maar mét Tailwind-voordelen:**

- combineerbaar met responsive modifiers
- combineerbaar met pseudo-classes



Dark mode

Tailwind ondersteunt dark mode via de variant prefix:

dark:*

Standaard volgt dit de systeeminstellingen, maar [manuele toggle is mogelijk](#) door een dark class op de tag te zetten (bijvoorbeeld via typescript).

app.css

```
@import "tailwindcss";  
  
@custom-variant dark (&:where(.dark, .dark *));
```

HTML

```
<html class="dark">  
  <body>  
    <div class="bg-white dark:bg-black">  
      <!-- ... -->  
    </div>  
  </body>  
</html>
```

@layer & @apply

tailwind op maat: CSS klassen

- Wat als je steeds dezelfde combinaties van tailwind utilities knipt en plakt? Er zijn verschillende opties.
- CSS regels en klassen zijn nog steeds een uitstekende oplossing
 - zeker als je iets snel wil kunnen aanpassen (DRY)
- Maar tailwind biedt ook extra functies

tailwind op maat: HTML element styles

- Tailwind [preflight](#) reset de style voor HTML elementen. voorbeeld: h1, h2... zien er hetzelfde uit, li heeft geen bullet...
- Wil je een eigen default style, zet die dan in tailwind's base @layer
- Er is ook een [typography plugin](#) met gestylede elementen ([playground](#)).
 - in voorbeeldcode: site/typo.html

css/components.css

```
@layer base {  
  h1 {  
    font-size: var(--text-3xl);  
    font-weight: var(--font-weight-bold);  
    color: var(--color-purple-800);  
    margin-bottom: --spacing(4);  
    @apply lg:underline  
  }  
  
  h2 {  
    font-size: var(--text-2xl);  
    color: var(--color-gray-800);  
    margin-bottom: --spacing(3);  
  }  
}
```

CSS var

tailwind functie

tailwind op maat: @apply

- @apply hergebruik tailwind utility klassen in CSS regels (kan niet in gewonen CSS)
- gebruik bij voorkeur gewone CSS met `var()` en `--functie()`

css/components.css

```
@layer base {  
  h1 {  
    font-size: var(--text-3xl);  
    font-weight: var(--font-weight-bold);  
    color: var(--color-purple-800);  
    margin-bottom: --spacing(4);  
    @apply lg:underline  
  }  
  
  h2 {  
    font-size: var(--text-2xl);  
    color: var(--color-gray-800);  
    margin-bottom: --spacing(3);  
  }  
}
```

CSS var

tailwind functie

tailwind utility klassen
apply-en

tailwind op maat: @layer components

- Als je CSS klassen in de component **@layer** zet kan je bij gebruik nog aanpassen met tailwind utilities

css/components.css

```
@layer components {  
  .card {  
    background-color: var(--color-green-100);  
    border-radius: var(--radius-lg);  
    box-shadow: var(--shadow-lg);  
    padding: --spacing(6);  
    margin: --spacing(1);  
    @apply border border-gray-300;  
  }  
  /* other component classes */  
}
```

tailwind op maat: @layer components

slides/components.html

```
<div class="card">  
  Season's greetings!  
</div>  
  
<div class="card border-dashed bg-gray-50 text-end">  
  When will I see you again?  
</div>
```

Season's greetings!

When will I see you again?

tailwind op maat: @layer utilities

- Voeg je eigen utility klassen toe

```
@layer utilities{  
  .flex-center {  
    @apply flex justify-center items-center;  
  }  
}
```

tailwind op maat: @layer utilities

- alternatief: kan ook op individuele utility met @utility

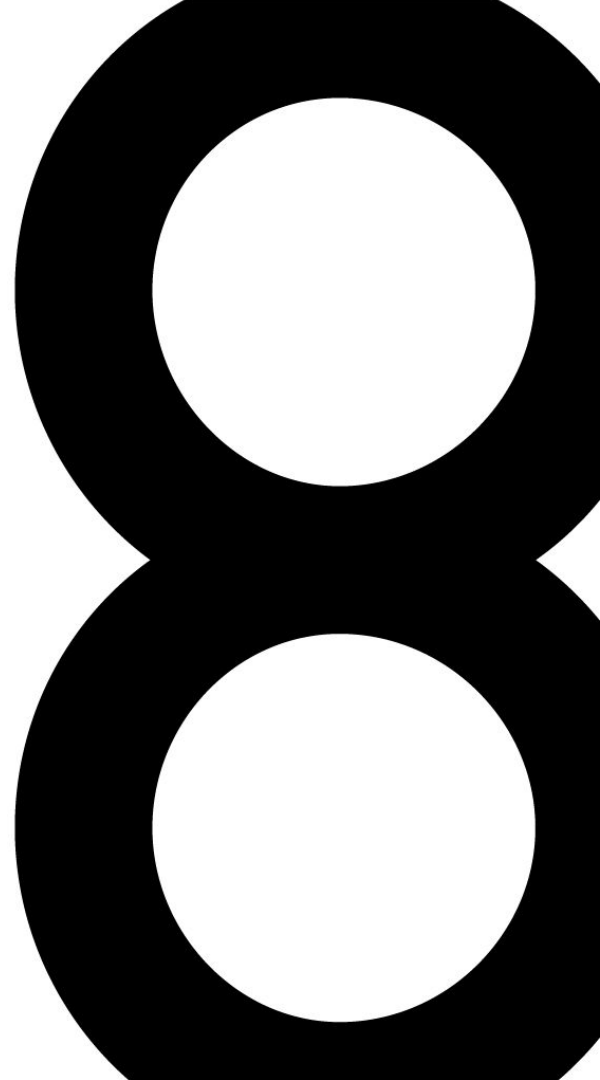
```
@theme {  
  --opacity-ghost: 0.7;  
  --opacity-shade: 0.3;  
}  
  
@utility opacity-* {  
  opacity: --value([percentage]);  
  opacity: calc(--value(integer) * 1%);  
  opacity: --value(--opacity-*);  
}
```

utilities kunnen ook een parameter hebben

als parameter geen % is, probeer om te zetten

als omzetten faalt probeer te matchen met --opacity- theme variable

Verder...



:has() variants (state-based styling)

Met de `has-*` variant kan je **een parent element stylen op basis van de state of content van zijn children**.

→ Tailwind's wrapper rond de CSS `:has()` pseudo-class

bv. `label` verandert als `input` aangevinkt is: `has-*` combineren met:

```
<label class="has-checked:bg-indigo-50
  has-checked:text-indigo-900
  has-checked:ring-indigo-200
  dark:has-checked:bg-indigo-950">
  Google Pay
  <input type="radio" checked />
</label>
```

```
// pseudo classes
has-[:focus]:ring-2

// element selectors
has-[img]:p-4
has-[a]:underline
```

group-has-*, peer-has-* & not-*

“Ik wil een child stylen op basis van iets anders in dezelfde component”

```
<div class="group">
  <p>Designer at <a href="#">planeteria.tech</a></p>
  <svg class="hidden group-has-[a]:block">
    <!-- icon -->
  </svg>
</div>
```

👉 SVG verschijnt **alleen** als er een **<a>** in de groep zit

zie ook:

- sibling state (**peer-has-***)
- **not-*** variant (inverse logic)

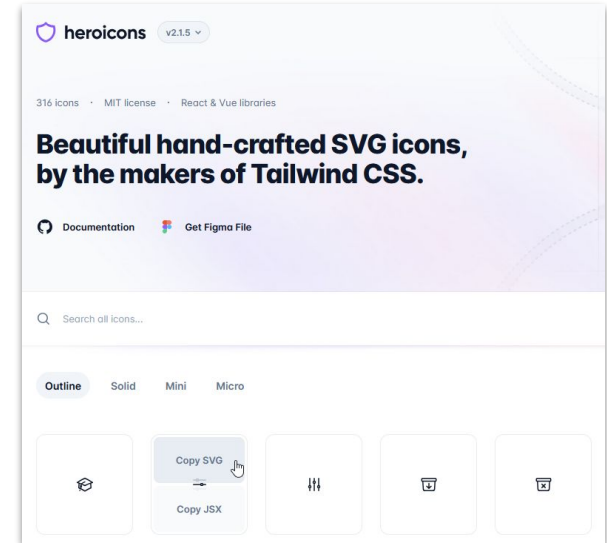


Heroicons

- svg icons die je naar je html kan kopiëren

Kan je stylen met utility classes

```
<svg xmlns="http://www.w3.org/2000/svg" fill="none"
viewBox="0 0 24 24" stroke-width="1.5" stroke="currentColor"
class="size-6 inline">
<path stroke-linecap="round" stroke-linejoin="round" d="..."
/>
</svg>
```





daisyUI

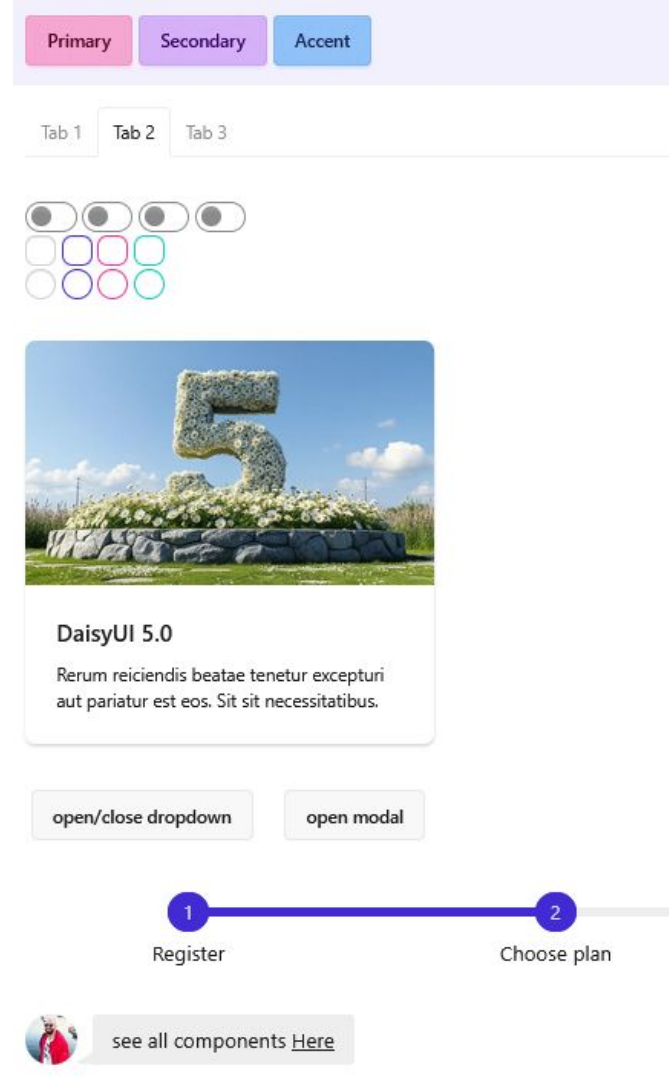


DaisyUI



daisyui.com

- [Library van componenten](#) gemaakt met tailwind
- [playground example](#)



Installatie (in tailwind project)

```
> pnpm i -D daisyui
```

app.css

```
@import "tailwindcss";
```

```
@plugin "daisyui";
```

- Een [skeleton project met tailwind + daisyUI](#) is beschikbaar

Componenten

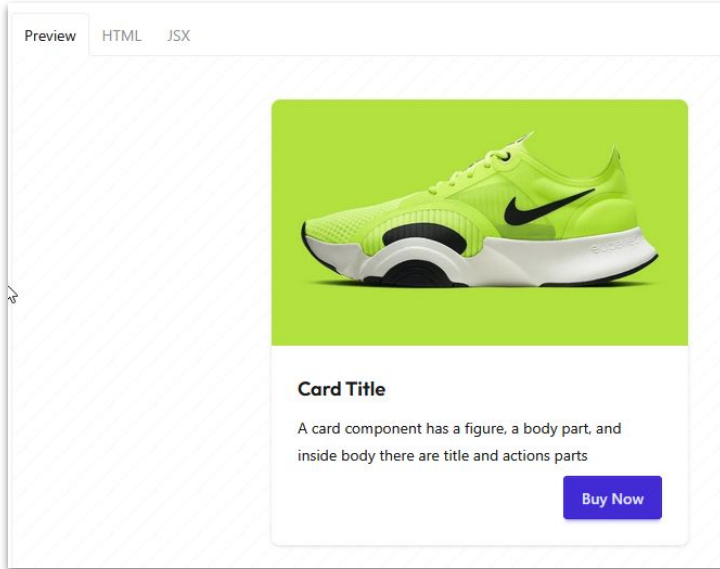
- Een component wordt gedefinieerd met een base class en helper classes
 - part, modifier, size, direction, placement, color...

The screenshot displays the Material Design Component Catalog interface. On the left, a sidebar lists various components, with 'Card' highlighted. The main area is titled 'Card' and includes a description: 'Cards are used to group and display content in a way that is easily readable.' Below this is a table detailing the components and their types.

prefix--	Class name	Type
	card	Component Card
	card-title	Part Title part
	card-body	Part Body part (content)
	card-actions	Part Actions part (buttons, etc.)
	card-border	Style Adds border to <card>
	card-dash	Style dash style
	card-side	Modifier The image in <figure> will be on to the side
	image-full	Modifier The image in <figure> element will be the background

Componenten

- Doc = learn by example: experimenteer



```
Preview HTML JSX
<div class="card bg-base-100 w-96 shadow-sm">
  <figure>
    
    </figure>
  <div class="card-body">
    <h2 class="card-title">Card Title</h2>
    <p>A card component has a figure, a body part, and inside body there are title and actions parts</p>
    <div class="card-actions justify-end">
      <button class="btn btn-primary">Buy Now</button>
    </div>
  </div>
</div>
```

Themes

app.css

```
@import "tailwindcss";
```

```
@plugin "daisyui";
```

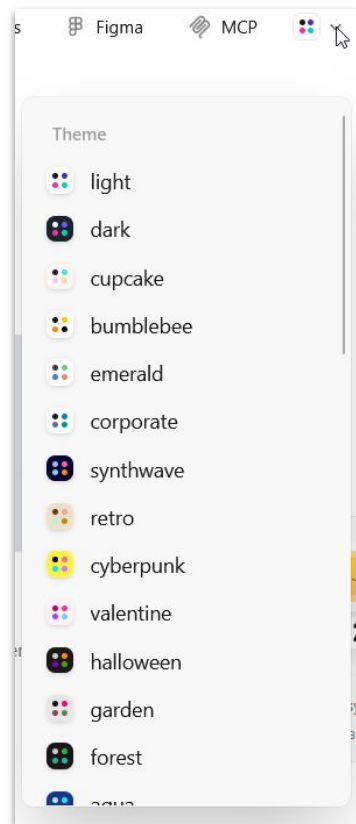
```
  themes: valentine --default, halloween --prefersdark;
```

```
}
```

stel **valentine** in als light theme
en **halloween** als dark theme

- Je kan je site theme configureren
- daisyUI bevat een aantal voorgebouwde themes
 - je kan ook themes toevoegen
- je kan themes via **data-theme** attribuut op je html overriden

```
<html lang="en" data-theme="cyberpunk">
```



Themes

- Voor daisyUI themes gebruik je semantische kleurnamen (-primary, -base,...) in plaats van de specifieke kleurnamen van tailwind
- Je gebruikt deze namen dan op de componentent:

```
<button class="btn btn-primary">Click me</button>
```

-  bg-primary
-  bg-secondary
-  bg-accent
- etc.